The Prompt Proposes, the Code Decides: Interface Rules for Small Quantized Code Agents

Roberto Vetere · Independent researcher · github.com/RobertoVetere

Abstract

How to connect small 4-bit models to tools as coding agents is largely unmeasured. We study Gemma 4 E4B and E2B, Qwen3-4B and Qwen3-1.7B, original and 4-bit, repairing Python repositories scored by hidden tests, under one principle: the model proposes the change; deterministic scripts locate code, apply edits and decide the next step. First, in a controlled test of 6,720 single calls on one server, a single long-text field was the only interface that stayed almost failure-free in every model (0–9%), while calls with several fields failed in up to 73%; how they fail is family-specific. Second, inside the agent, the same E4B with the same scripts had 286 of 621 calls rejected with a three-field call and 0 of 687 with one-field commands. Third, 4-bit weights left E4B's isolated calls unchanged but amplified its call failures under long context; they amplified failures more for the smaller E2B, and not for Qwen3-4B. Fourth, reasoning helped a free-form agent more than a script-guided one with a 31B, an effect that shrank on replication and vanished with 4-bit E4B. Hypotheses for the generalization runs were registered before running them. We condense the evidence into five rules and release agents, data and a validation set.

1. Introduction

Coding agents are usually designed for large models behind commercial APIs. A small local model with quantized weights writes tool calls less reliably, edits less precisely, and spends its time budget on the tokens it generates.

We start from a simple principle: *the prompt proposes, the code decides*. The model chooses what to change; deterministic scripts find the code, apply the edit and run checks, and their verdict decides the next step. The principle comes from an earlier prompt-chaining tool in which splitting each problem into a solving call and a separate strict-answer call, whose output code reads, raised a 0.5B model from 25% to 62.5% on 16 arithmetic problems (a single run).

Our contributions: (1) a controlled test across four small models from two families, with original and 4-bit weights, showing that one long text per call is the only interface that keeps tool calls intact; (2) an A/B inside the agent that isolates the interface as the cause of format failures; (3) quantization effects measured on one server, with and without long context; (4) four evaluation pitfalls, including an API that silently served another model; and (5) a validated task set, tools to run the official evaluator outside the competition, and a notebook that regenerates every table. Hypotheses for the runs beyond E4B were registered in the repository before running them.

2. Experimental setup

Task. Each task is a Python repository at a fixed commit, an issue text, and hidden tests that the fix must make pass or keep passing. The agent runs in a sandbox without network, with limits of 8 minutes, 28 tool calls and 80 model turns.

Environment. We use the evaluation environment of the Gemma 4 Developer Agent competition [1]: agents declared in YAML on Google's Agent Development Kit (ADK) [2], nine predefined tools, and custom code only as Python scripts in a *skill*, invoked with a three-field call. The framework periodically replaces earlier turns with a model-written summary (compaction interval 15); the system instructions remain, but earlier tool calls and outputs do not.

Models. Official 4-bit weights (Gemma 4 QAT w4a16 [3], Qwen3-4B AWQ) and original bf16 weights, served by us with vLLM 0.19.1 [4] and each family's tool-call parser, as the competition serves Gemma. A commercial API (DeepInfra) answered our requests for E4B with `gemma-4-31B-it-turbo`, as we found after the fact (4.6); those results are reported as 31B (API). Temperature is 0.2.

Task sets. The *validation set* has 104 tasks from 24 repositories of a public CC BY 4.0 collection [5] whose reference patch passes in the competition environment; agent comparisons use 40 sampled at random from the 65 that also pass outside the competition environment. The competition's *test set* has 58 held-out tasks, seen only as an aggregate score.

3. Method

Our agent, `fixer`, has three tools (`run_command`, `write_file`, `submit_patch`) and one skill. Its only skill call copies the scripts into a git-ignored folder in the repository; after that, every step is a one-field shell command, and long text (the issue, test cases, the edit) goes through `write_file`. Steps: `evidence.py` lists candidate functions from tracebacks, quoted commands and issue strings; `show_function.py` shows a function and what it calls; `callers.py` lists its callers; the model writes test cases; `apply_edit.py` applies a search-and-replace edit or shows the closest real lines; `verify.py` runs the cases and a related test and returns `SUBMIT`, `FIX` or `NO_EDIT`. Every script ends with a `NEXT:` line giving the exact next command, because the most recent tool output stays visible after a summary.

4. Experiments and results

4.1 Tool calls fail in predictable ways

Across the traces of successive agent versions run in the competition environment, five failures recur:

Failure	Cause	Remedy
The skill is called by the agent's name	The framework injects the agent's name into every request	Give agent and skill the same name
The skill does not load	Only lowercase names with hyphens are accepted	Validate names before running

Failure	Cause	Remedy
The call is written as text and never runs	The model imitates the prompt's code-like notation	Describe calls as lists of fields
Fields are lost when an argument is long	Long text inside a multi-field call (4.2)	One long text per call, through files
The same incomplete call is repeated 64 times	The summary drops the earlier calls that worked	One-field commands and `NEXT` lines

With the three-field skill call the 31B had 819 of 930 calls rejected (653 from the naming failure; 81% after fixing it). These traces compare successive versions; 4.2 and 4.3 test the cause directly.

4.2 A controlled test across four small models

We asked each model to apply one edit with one tool call, copying an old and a new block of real code (6–30 lines, 80 blocks from the validation repositories, two samples each), through three interfaces: the skill call with the text nested in an `args` object, a flat `edit_file(path, old_string, new_string)`, and a single `content` field holding a search-and-replace block; with or without 10,000 tokens of previously read code before the request. A call fails structurally if it never runs (no call the parser can extract) or loses a required field.

Structural failures of 160, short / long context	Nested	Flat	One field
Gemma 4 E4B, bf16	7 / 6	0 / 0	0 / 0
Gemma 4 E4B, 4-bit	4 / **21**	0 / 0	0 / 0
Gemma 4 E2B, bf16	41 / 68	10 / 0	0 / 0
Gemma 4 E2B, 4-bit	87 / **105**	35 / 27	0 / 2
Qwen3-4B, bf16	103 / 101	99 / 84	0 / 0
Qwen3-4B, 4-bit (AWQ)	116 / 100	92 / 79	2 / 8
Qwen3-1.7B, bf16	9 / 113	30 / 74	1 / 15

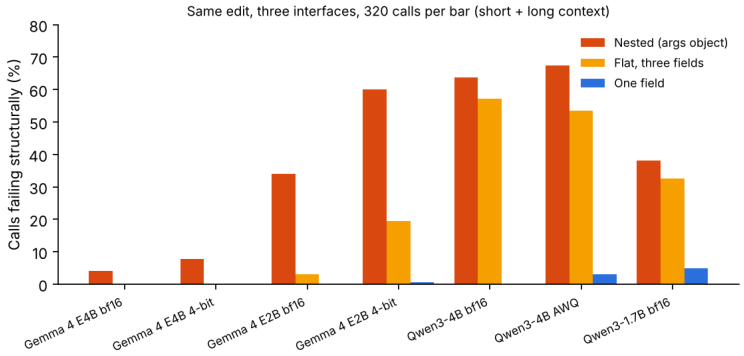


Figure 1: share of calls failing structurally per model and interface (320 calls per bar).

The single field never failed more than a multi-field interface: it failed less than the nested call in every model (block-level McNemar, $p \le 0.008$) and less than the flat call wherever that one failed often (E2B 4-bit and the Qwen models, $p < 10^{-9}$). Several

registered predictions failed: nested calls were not always worse than flat ones; flat and single-field calls stayed under 2% only for E4B (single fields reached 9% for Qwen3-1.7B with long context); and the extra harm of 4-bit weights under long context held only for E4B. How multi-field calls fail depends on the family: Gemma drops `skill_name` or `file_path` or absorbs them into `args`; Qwen writes the call as text that its parser cannot extract. The 4-bit weights amplified failures of the nested call for E4B only under long context (12 against 4 of 80 blocks, $p = 0.04$) and for E2B in both contexts ($p < 0.001$), but not for Qwen3-4B AWQ. Single fields kept their structure but did not always keep the requested layout: E2B wrote the edit without the search-and-replace markers in most calls, which code can detect and report.

4.3 The same agent with two interfaces

We ran two versions of the agent with 4-bit E4B on the 40 tasks. They share prompt steps and scripts; one calls every script through the three-field skill call, the other installs once and then uses one-field commands.

E4B 4-bit, 40 tasks	**Calls**	**Rejected for format**	**Solved**
Three-field skill calls	621	286 (46%)	8
One-field commands	687	0	9

The interface alone removed every format rejection, as registered (H3); the solved tasks were nearly the same (one task solved only with one-field commands), so reliable calls are necessary but not sufficient.

4.4 Reasoning and agent structure

We crossed `fixer` with a free-form public agent that edits files with the built-in tools, with and without reasoning (4,096 tokens), on the 40 tasks, three times through the API (31B) and twice with 4-bit E4B on our server.

Solved of 40, per replica	**No reasoning**	**Reasoning**
Free-form, 31B (API)	14, 13, 14	20, 17, 15
`fixer`, 31B (API)	16, 16, 15	10, 16, 15
Free-form, E4B 4-bit	12, 12	13, 14
`fixer`, E4B 4-bit	9, 12	12, 10

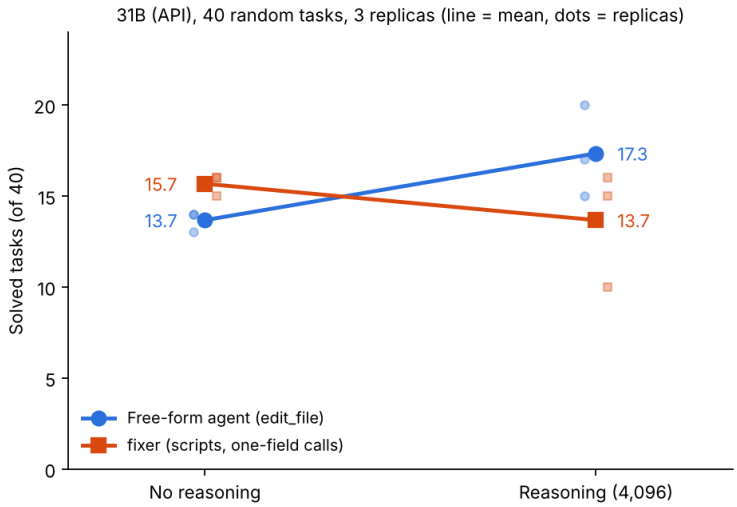


Figure 2: 31B (API), 40 tasks; line = mean of three replicas, dots = replicas.

With the 31B, reasoning helped the free-form agent in every replica and, in the first run only, hurt `fixer` (per-task interaction 11 against 1, p = 0.006; 8 against 4 in the two new replicas, p = 0.39); with reasoning, `fixer` hit the 80-turn limit in 5, 5 and 4 tasks against 0, 0 and 1. With 4-bit E4B the interaction vanished (3 against 3 tasks): reasoning helped the free-form agent slightly, as registered, but did not hurt `fixer`, contrary to our registered prediction. Reasoning is not a free gain, and its value depends on model and design.

4.5 Quantization in isolated calls

On 11 tasks, two samples each, single calls located the code from a repository map and wrote an edit scored with the hidden tests. E4B bf16 and 4-bit solved 6 and 5 of 22 without reasoning, 5 and 5 with reasoning; generated 2,094 and 2,002 reasoning tokens (median); lost 18 and 24 of 88 edits to miscopied search text (p = 0.38); and located the right symbol in 30 and 29 of 40 tasks. The 31B (API) generated 6,002 tokens and solved 9 of 22 with reasoning. For E4B, quantization's cost appeared only in tool calls under long context (4.2). Localization was not the bottleneck either: a model-free retriever placed the changed function or its class in its top three for 28 of 40 tasks, and with the right file the 4-bit E4B still failed 55 of 64 repair attempts.

4.6 Evaluation pitfalls

The API may serve another model. Our requests for E4B were answered by a 31B: the provider had marked the model as replaced, our bridge logged the requested name, and the bill matched 31B prices to the cent. Only the response's `model` field showed it.

Tuning tasks inflate success. A 31B agent solved 7 of the 8 tasks used to design it but 14 of 40 random ones.

Runs do not repeat. At temperature 0 with a fixed seed, 0 of 8 trajectories were identical across two runs, likely because batched inference is not batch-invariant [6]; a first-run effect shrank on replication (4.4).

Local success does not transfer. The 4-bit E4B solved 9 to 14 of the 40 validation tasks on our server (4.4), but its agents scored 0.01 and 0.03 on the competition test

set. Weights and vLLM version match the competition's, so the gap lies in the tasks, whose repositories and descriptions both differ: the competition's 72 development tasks (the test tasks are hidden) are short pull-request descriptions, median 72 words against 108 in validation, none with a runnable example.

5. Related work and convergent evidence

SWE-bench [7] defined repository repair with hidden tests; SWE-agent [8] showed that the agent–computer interface matters, and Agentless [9] that a fixed localize–repair–validate pipeline can beat autonomous agents. We study the other end of the scale: an interface that a small quantized model cannot call incorrectly. Strict output formats degrade reasoning [16], and reasoning models overthink in agentic repair [17]. 4-bit weights are near-lossless on reasoning benchmarks [18], yet in agents they amplify failures the model already has, up to 2.5 times, with an unchanged score [19], and they nearly triple missed tool calls in Qwen3-1.7B but not in Qwen3-4B [20]; we likewise see amplification for E4B under long context and for the smaller E2B, and none for Qwen3-4B. Other participants report agreeing results:

Finding	**This paper**	**Independent evidence**
Interfaces steer; instructions do not	286 of 621 vs 0 of 687 rejected (4.3)	Prompt rules rarely followed; interface changes worked [10]
Long text in structured calls breaks	Four models, two families (4.2)	Single-string edits cut failures from 56% to 2% [11]
Reasoning is not a free gain	Depends on model and agent (4.4)	Keeping or dropping earlier reasoning changed nothing [13]; thinking did not help in the time budget [14]
Localization is not the bottleneck	Retriever top 3 for 28 of 40	Name lookup drives localization [15]
One run is one sample	0 of 8 identical; effects shrink	One leaderboard draw varies by about 1.5 tasks [12]

6. Limitations

Agent comparisons use 40 tasks; the 31B came through an API with an unknown serving configuration; the format test copies given text rather than writing new code; only Gemma had QAT weights; some pre-registered hypotheses failed and are reported as such (`docs/HYPOTHESES_12NOV.md`).

7. Conclusion

For small quantized models, the interface decides whether an agent works, not how much it solves. Five rules:

1. **One long text per call.** Put it in a single field or a file; nested multi-field calls with long text failed in every small model we tried, and flat ones in most.
2. **Put the next step in the tool output**, which stays visible when a summary replaces earlier turns.

3. **Let code decide and check layout.** Scripts locate, apply and verify, and reject malformed edits.
4. **Measure reasoning per model and design.**
5. **Validate with the deployed weights and tasks like the target**, with repeated runs, and check which model answered.

No rule fixed the content of the change, where most failures remain.

Reproducibility

Code, agents, data and per-call and per-task results are at https://github.com/RobertoVetere/nervo-gemma-paper (tag `paper-v1`). `docs/REPRODUCE.md` lists the file and command behind every number; `notebooks/reproduce_paper.ipynb` regenerates every table on a CPU. The retriever of Section 4.5 and the agents built on it will be added after the competition closes (2 December 2026).

References

[1] E. Markowitz, B. Perozzi, B. Rózemberczki et al. *Google – The Gemma 4 Developer Agent Competition.* Kaggle, 2026. https://www.kaggle.com/competitions/gemma-4-developer-agent

[2] Google. *Agent Development Kit (ADK).* https://google.github.io/adk-docs/

[3] Google. *Gemma 4 quantization-aware trained checkpoints (w4a16).* Hugging Face, 2026.

[4] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

[5] D. Gluzdov. *gemma-4-task-soundness-and-local-results* (dataset, CC BY 4.0). Kaggle, 2026.

[6] H. He and Thinking Machines Lab. Defeating Nondeterminism in LLM Inference. *Thinking Machines Lab: Connectionism*, 2025.

[7] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.

[8] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.

[9] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. FSE 2025. arXiv:2407.01489

[10] hassantavakoli. *Instructions lose to interfaces: what steers Gemma 4 31B as a coding agent.* Kaggle, 2026. https://www.kaggle.com/code/hassantavakoli/instructions-lose-to-interfaces-reproduce-tables

[11] M. M. Valiyavalappil et al. *Action-Masked LoRA and Empirical Failure Modes in Gemma 4 SWE Agents.* Kaggle, 2026. https://www.kaggle.com/code/mrmaheshv/action-masked-lora-swe-agents

[12] S. M. Hossain. *Why Your Leaderboard Can’t See a 3-Point Gain.* Kaggle, 2026. https://www.kaggle.com/code/mobassir/why-your-leaderboard-can-t-see-a-3-point-gain

[13] hemantkumarbk. *Overflow is a symptom, not a cause* (ThoughtKeep). Kaggle, 2026. https://www.kaggle.com/code/hemantkumarbk/thoughtkeep-reproduction

[14] J. J. Harris. *Declarative Agents for Autonomous Software Engineering: A Gemma 4 Case Study.* Kaggle, 2026. https://www.kaggle.com/code/jefferyjeromeharris/gemma-4-dev-agent-paper

[15] lizhang20252026. *Nodes or Edges? What a Code Graph Actually Contributes to a Small Coding Agent.* Kaggle, 2026. https://www.kaggle.com/code/lizhang20252026/nodes-or-edges-tool-ablation-notebook

[16] Z. R. Tam et al. Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models. EMNLP 2024 (Industry).

[17] A. Cuadron et al. The Danger of Overthinking: Examining the Reasoning-Action Dilemma in Agentic Tasks. arXiv:2502.08235, 2025.

[18] R. Liu et al. Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models. arXiv:2504.04823, 2025.

[19] J. Jang, K. Yang, H. Lim, H. Park. Flat Score, Amplified Failures: How the Error Budget Masks Damage in Quantized LLM Agents. arXiv:2607.27275, 2026.

[20] M. Roshaan, M. Aaliyan, S. Bin Zubair. Compression Breaks Discipline: Quantization Increases Missed Tool Calls in Small Language Models, but Not Mid-Size Ones. NeurIPS 2026 Workshop.